Assignment 3: Information Traversal and Node Vulnerability: A Linked List Perspective on "Memento"

林柏翰
1113405021
國立澎湖科技大學

***Abstract*—本報告探討電影《記憶拼圖》(Memento) 主角 Leonard 的碎片化記憶系統與「鏈結串列」(Linked List) 資料結構之間的映射關係 [cite: 181, 182]。透過分析其依賴照片與指標 (Pointer) 進行資訊尋訪的過程，探討其在時間複雜度與資料完整性維護上面臨的巨大挑戰。本研究將反思連續記憶體對於高效能資料檢索的重要性，並探討指標竄改對系統安全性造成的毀滅性影響。**

***Index Terms*—Linked List, Sequential Access, Pointer, Memento, Data Integrity, Time Complexity, Memory Model.**

I. Introduction

在電腦記憶體架構中，陣列 (Array) 憑藉著連續的記憶體配置 (contiguous memory)，擁有能在 $O(1)$ 常數時間內達成隨機存取 (Random Access) 的強大優勢 [cite: 101]。人類正常的短期與長期記憶運作，在某種程度上也具備類似的高效索引能力。然而，與陣列不同，鏈結串列 (Linked List) 是一種 pointer-based (指標式) 的動態資料結構 [cite: 89]。它由多個非連續節點 (Node) 串接而成，不需要連續記憶體 [cite: 90, 91]。電影《記憶拼圖》主角 Leonard 因患有順向失憶症，無法建立長期記憶，記不住超過幾分鐘的事情 [cite: 163]。他的大腦資料結構被迫降級，必須透過實體的照片、筆記與刺青來建立資訊連結 [cite: 169]。這種極度依賴「下一個線索」來導向最終目標的行為模式，與 Linked List 的指標機制不謀而合。本報告將深度剖析這種結構在現實應用中的檢索成本與脆弱性。

II. 記憶的節點 (Node) 與指標 (Pointer) 映射結構

Linked List 的本質是由一系列分散在記憶體各處的節點所組成，每個節點 (Node) 僅包含兩項核心資訊：資料 (data) 與指向下一個節點記憶體位置的指標 (next) [cite: 93, 94]。因為記憶體位置不連續 [cite: 97]，我們無法得知整個串列的全貌，只能依賴指標單向尋訪。Leonard 的記憶系統完美映射了這個結構 [cite: 169]：

- **Node (資料節點)**：他每一次拍下的拍立得照片，或是寫在紙條上的零碎文字，甚至刺在身體各部位的刺青，都是一個獨立的 Data Node [cite: 169]。這些節點在時間與空間上是極度破碎且不連續的。
- **Next Pointer (指標)**：照片背後的註記 (例如：「這是 Natalie，她也會幫你」) 或刺青上的指令，就是 `next` 指標 [cite: 169]。它本身並不包含真相的全部，只是無情地將主角的注意力導向「下一個記憶體位址」。最後一個線索會導向復仇的終點 (如同指向 null) [cite: 95]。
- **Head (起始點)**：每當 Leonard 的記憶被強制重置 (Cache 被清空) 醒來時，他總是處於茫然狀態。他必須先摸索口袋裡的第一張照片，或是照鏡子看到胸口最大的刺青，從這個點開始找起 [cite: 169]。這個動作，等同於程式碼中初始化走訪的指令：`Node current = head;`，這是一切運算與尋訪的開端。

III. 序列走訪 (Sequential Traversal) 的極高時間成本

由於 Leonard 失去了如同 Array 般 $O(1)$ 的直接存取能力 [cite: 101]，他的生活與復仇之路充滿了巨大的延遲與無力感。在 Linked List 中，若要尋找特定的資料 (Search)，演算法必須從 `head` 節點開始，執行循序存取 (Sequential Access)，一路往前查 [cite: 169]，逐一比對每個節點，直到找到目標為止。這種逐個走訪的搜尋時間複雜度高達 $O(n)$ [cite: 102, 157]。

在電影中，我們無數次看到主角醒來後，必須耗費大量時間焦慮地翻找口袋、查看刺青、閱讀筆記。他必須將線索 A 連結到線索 B，再由 B 推導到 C。這個 $O(n)$ 的檢索過程不僅消耗了極大的時間與運算資源，更悲哀的是，由於他的暫存記憶體 (短期記憶) 容量極小，經常在執行走訪到一半時，系統就發生了中斷 (Interrupt)，導致走訪進度完全歸零。他又必須無奈地再次回到 `head` 重新啟動搜尋流程。這凸顯了在缺乏 Cache locality (快取區域性) 的情況下 [cite: 99]，Linked List 處理頻繁查詢任務時的極度低效。

IV. 指標竄改與資料結構的極端脆弱性 (Vulnerability)

Linked List 最具優勢的地方在於其插入 (Insertion) 與刪除 (Deletion) 操作非常迅速，只需修改 pointer 而不需搬移龐大的資料 [cite: 103, 105, 135]。但諷刺的是，這個原本的演算法優勢，卻成為 Leonard 記憶系統中最致命的安全弱點。

因為系統使用非連續記憶體配置，且缺乏總體的校驗機制 (Checksum)，外部人士可以非常輕易地「竄改指標」。電影中的 Teddy 與 Natalie，正是利用了這個漏洞。他們只需要在 Leonard 的某張照片背後寫下虛假的提示，或是藏起某張關鍵照片，就等同於在 Linked List 的中段強制執行了 $O(1)$ 的 `insert()` 或 `delete()` 操作 [cite: 158]。當指標 `node.next` 被惡意重新指向錯誤的記憶體位址時，

整條資訊鏈就會完全走樣。Leonard 毫無防備地順著被竄改的指標前進，最終將無辜的人視為仇人。這顯示了在缺乏雙向驗證（如 Doubly Linked List）或資料加密的情況下，單向鏈結串列在面對惡意篡改與邏輯攻擊時是何等脆弱。

V. Conclusion

《記憶拼圖》不僅是一部結構精巧的懸疑片，更是一場關於資訊鏈路完整性（Data Integrity）的殘酷實驗。透過 Linked List 的觀點，我們能深刻體會到當一個系統失去「連續記憶體」的底層支撐與「快速存取」的索引能力時，資訊的檢索將變得極其低效 $O(n)$ 且極度脆弱。主角的悲劇源自於其大腦硬體受損後，被迫採用了安全性極低的動態資料結構來處理攸關生死的決策。這強化了我們在設計現代資訊系統時，對於資料保護、存取路徑優化以及指標安全防護的重視；設計演算法絕不能僅考慮最佳情況，更需嚴防最壞情況下的系統崩潰。